Multi-Agent Architecture · Day 1

# Why one AI agent is rarely enough

Complex work improves when specialized agents collaborate instead of one generalist doing everything.

Autonomous Systems Workshop · Foundation Series



Made with GAMMA

System Architecture

# Multi-agent systems divide the work by strength

01 Focus

## Specialization

Each agent focuses strictly on what it does best, maintaining sharp context and precise domain instructions without distraction.

02 Concurrency

## Parallel processing

Independent work streams execute concurrently across separate tools, dramatically cutting total runtime on unblocked tasks.

03 Resilience

## Robustness

Other agents compensate, critique, and recover when an individual task fails, isolating errors before they corrupt the pipeline.

**Core insight**

Generalist models dilute context; specialized agents bound scope and enforce deterministic evaluation.

Made with GAMMA

Analogy

# A newsroom is the mental model

Reporter gathers facts, writer shapes the story, editor checks the result, and designer creates the visual layer.

1. **Reporter**
   Gathers facts.
2. **Writer**
   Shapes the story.
3. **Editor**
   Checks the result.
4. **Designer**
   Creates the visual layer.

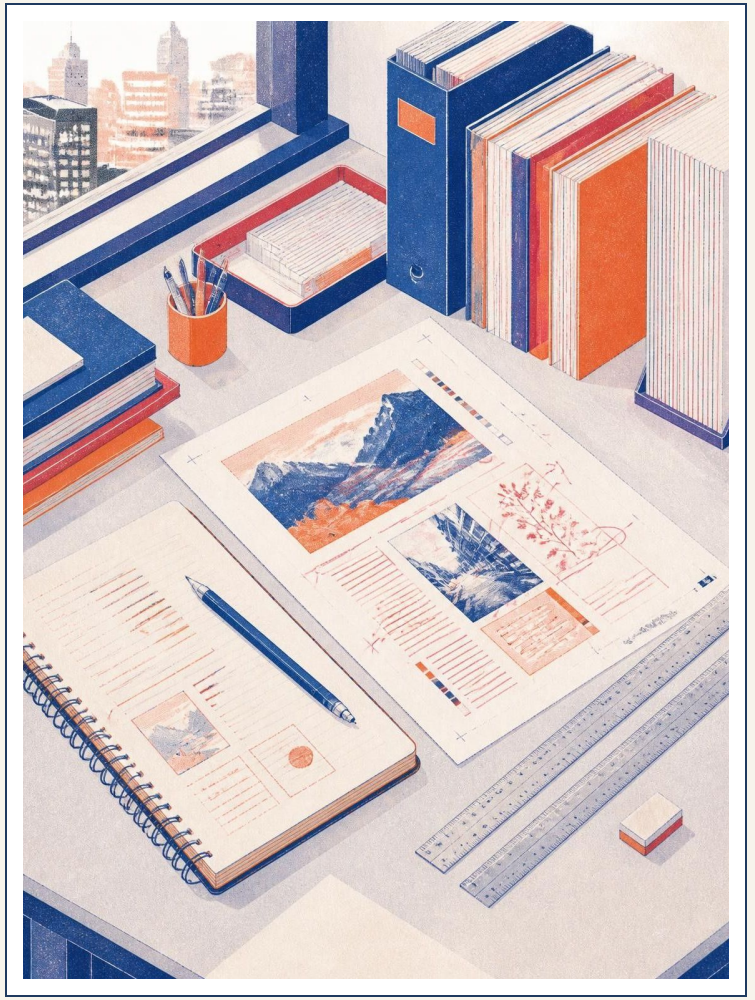


Fig. 1.1 The classic editorial desk workflow

Made with GAMMA

Execution Pipeline

# The workflow is a chain of specialized handoffs

Stage 01

## Researcher

Finds reliable information.

Stage 02

## Writer

Turns findings into content.

Stage 03

## Editor

Refines clarity, grammar, and style.

**Each agent stays** in its optimal context while the output moves forward.

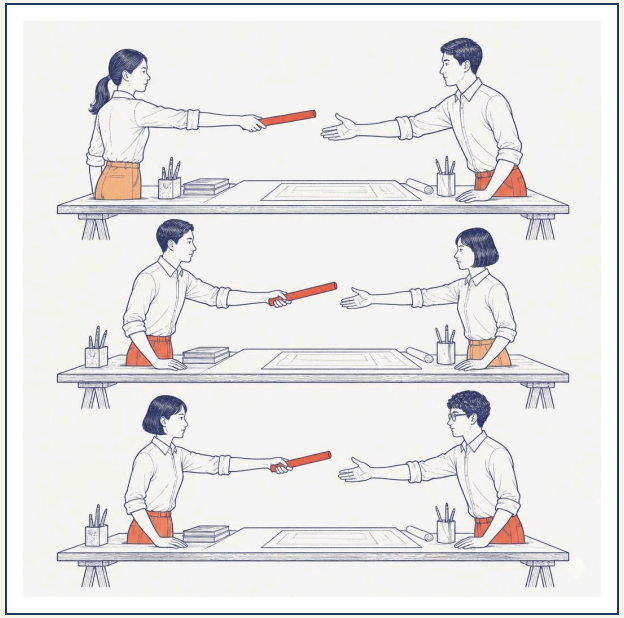


Fig. 1.2 Hand-off integrity across stages

Made with GAMMA

Framework Abstraction

# CrewAI turns coordination into a framework

1. Define agents, tasks, and dependencies
2. Pass context between tasks automatically
3. Run the crew and inspect a clear execution flow

The developer focuses on the work design instead of hand-building orchestration.

Made with GAMMA

Core Taxonomy

# Four building blocks explain the whole system

Primitive 01

## Agent

The specialized worker equipped with identity, domain focus, tools, and reasoning capabilities.

Primitive 02

## Task

The concrete unit of work containing clear objectives, input context, and strict expected outputs.

Primitive 03

## Crew

The coordinating entity that binds agents and tasks together to produce the end result.

Primitive 04

## Process

The execution strategy (sequential or hierarchical) deciding how tasks hand off and resolve.

**Mental model**

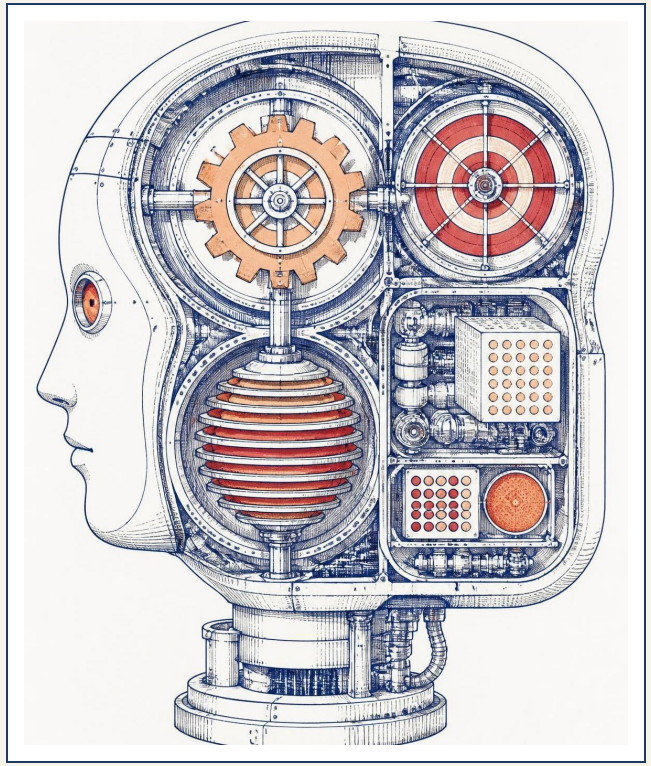
Agents do the work, Tasks specify the work, Crews assemble the team, and Processes govern the flow.

Made with GAMMA

Anatomy of an Agent

# An Agent brings role, goal, and capability

- **Role:** what job the agent performs
- **Goal:** what outcome it is trying to achieve
- **Backstory:** the expertise and personality that guide decisions
- **Tools and LLM:** what it can use to complete the work



Made with GAMMA

# A Task makes the work concrete

1

## Description

Description of what needs to be done

2

## Assigned agent

Assigned agent responsible for the work

3

## Context

Context from earlier tasks when dependencies exist

4

## Expected output

Expected output that defines “done”

Made with GAMMA

System Assembly

# A Crew connects people, work, and workflow

## Crew.kickoff()

The crew holds the agents and tasks, then applies a process to coordinate execution and return a final result.

### 1. Ingest Agents

Validates roles, tools, and models.

### 2. Schedule Tasks

Resolves dependency DAGs.

### 3. Return Output

Delivers structured final artifacts.

### Orchestrator Role

Manages memory, error handling, token limits, and task dispatching behind a unified interface.

### Single Entry Point

The client app interacts with one simple call: pass runtime parameters to `kickoff()` and await output.

Made with GAMMA

Pattern Selection

# Choose the process that matches the dependency pattern

## Sequential Process

Predictable linear handoffs, one task after another.

| | | |
|---|---|---|
| **Pattern** | Deterministic pipeline (A → B → C) | |
| | **Control** | Tasks execute in strict declaration order |
| **Best for** | Dependent tasks where output is strict input | |

## Hierarchical Process

A manager agent delegates and coordinates workers.

| | |
|---|---|
| **Pattern** | Adaptive routing via managerial LLM |
| **Control** | Manager delegates, reviews, and synthesizes |
| **Best for** | Complex tasks requiring parallel review & iteration |

**Rule of thumb**

Start with Sequential for predictable reliability; step up to Hierarchical when dynamic planning is required.

Made with GAMMA